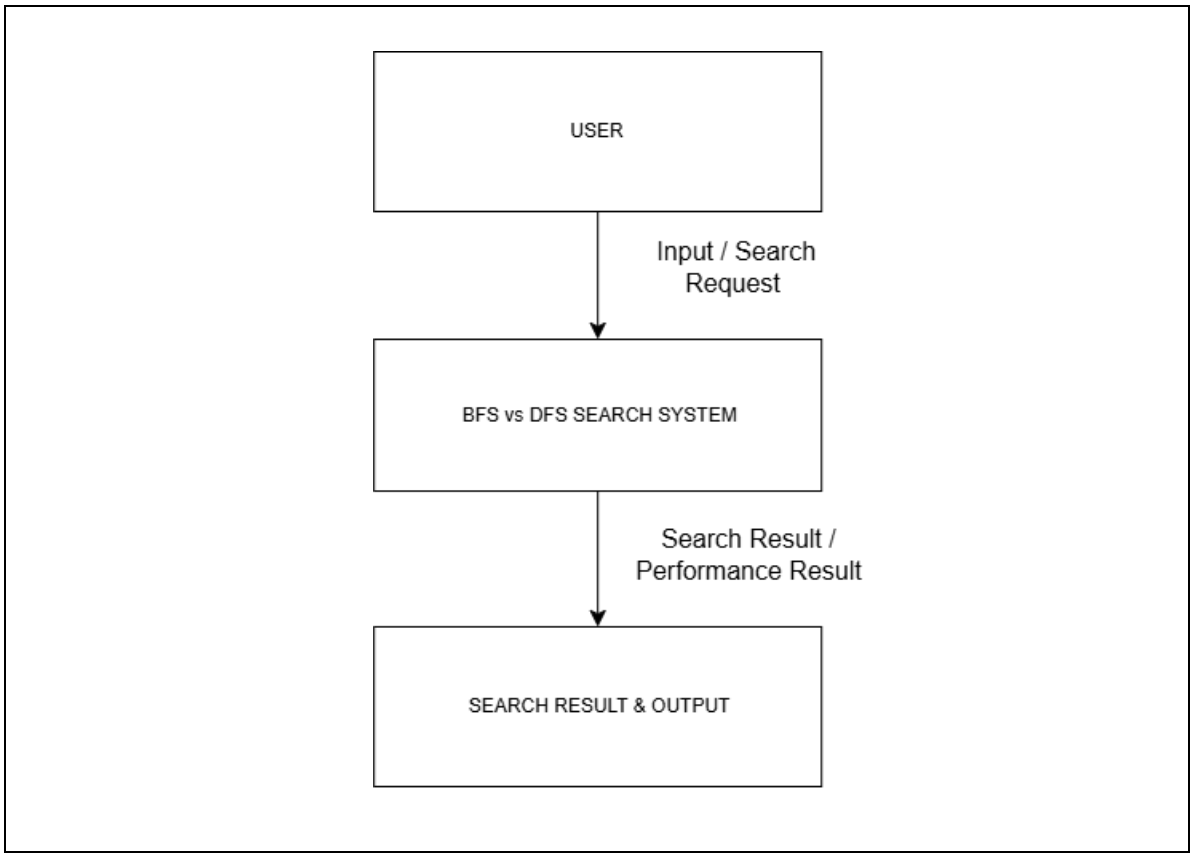
SLE	SLE-3: Architectural Design (Full C4 Model)
Course	02AML204 – Introduction to Artificial Intelligence
PRN	25UAM037
Name	Pallavi Vasant Koli
Division	A
GitHub / Portfolio Link	https://github.com/pallavikolii285-ops/IAI_SLE3.git

1. System Title & Short Description:

   BFS vs DFS Search System is a Python-based graph search system that implements Breadth First Search (BFS) and Depth First Search (DFS). The system searches for a goal node from a given start node in the same graph. BFS uses a queue, while DFS uses a stack for exploring nodes. The system also measures execution time and counts expanded nodes for performance comparison.

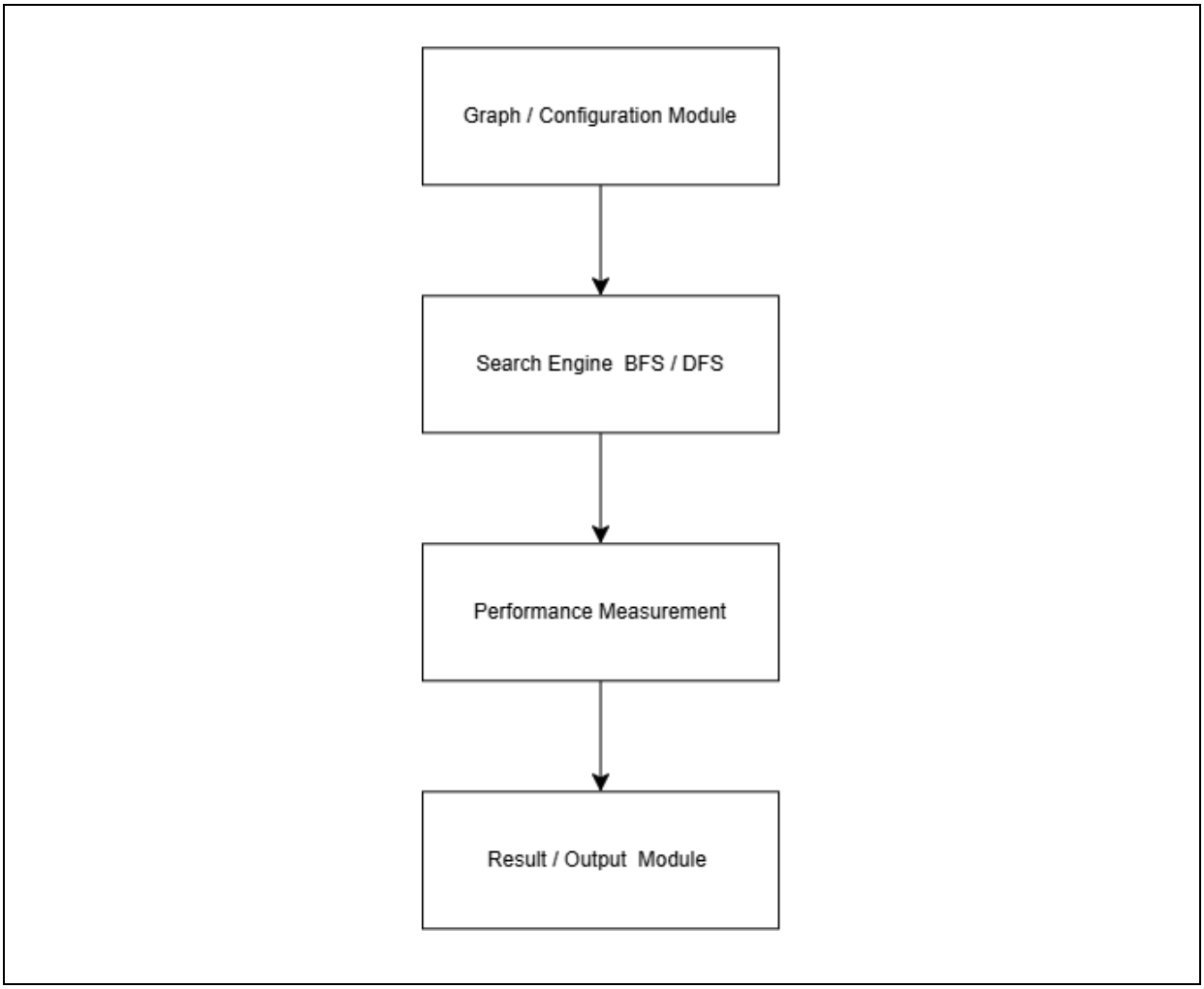
2. Context Diagram (C4 Level 1) :



The user provides the search input and starts the BFS or DFS search. The BFS vs DFS Search System processes the graph using the selected search algorithm.

The system searches from start node 0 to goal node 19. The output provides the search result and performance information.

3. Container Diagram (C4 Level 2) :



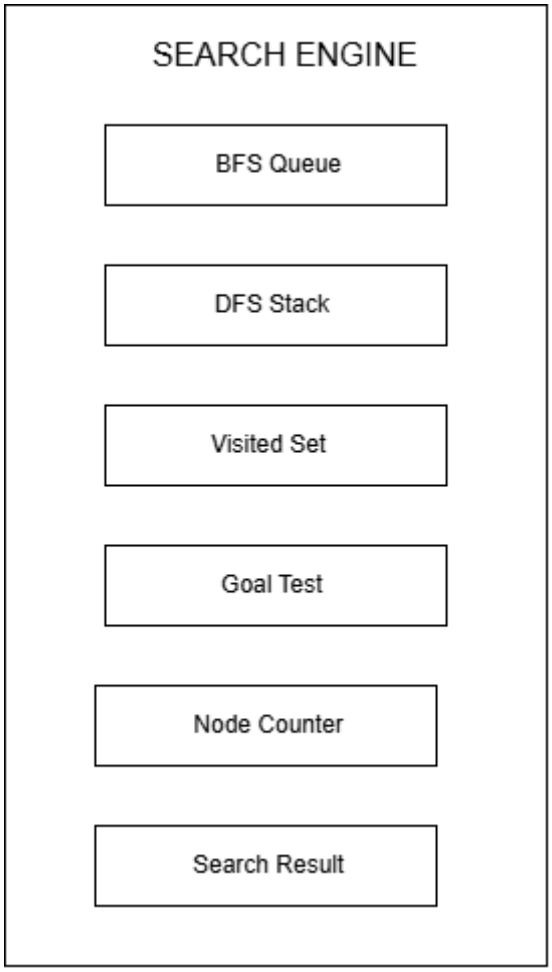
Graph / Configuration Module:
Stores the graph and search configuration such as start node 0 and goal node 19.

Search Engine:
Executes BFS and DFS algorithms on the same graph using their respective data structures.

Performance Measurement:
Measures the execution time of the search algorithms using Python's `timeit` module.

Result / Output Module:
Displays the search status, execution time and number of expanded nodes.

4. Component Diagram (C4 Level 3) :



The Search Engine is divided into components that support BFS and DFS execution. BFS uses a queue and DFS uses a stack to manage nodes waiting for exploration. The visited set prevents repeated node exploration. Goal Test checks whether the required goal node is reached, while Node Counter records the number of expanded nodes.

5. Code Level Overview (C4 Level 4) :

Main Class / Function	Responsibility
`graph / configuration`	Defines the graph and search settings.
`bfs()`	Performs Breadth First Search using a queue.
`dfs()`	Performs Depth First Search using a stack.
`timeit.default_timer()`	Measures the execution time of the search.
`Average Calculation`	Calculates average performance values.
`print()`	Displays the search and performance results.

6. Design Decisions:

The same graph, start node and goal node are used for both BFS and DFS to maintain a consistent search setup. The Search Engine is selected as the main container because it contains the core search operations. Queue and stack are represented as separate components because BFS and DFS use different search strategies.

7. AI Contribution Note:

AI tools used:
ChatGPT

What AI helped with:
Understanding the C4 model, organizing the architecture and preparing the diagram structure.

What I did myself:
I used my SLE-2 BFS/DFS system, checked the repository, understood the code structure and prepared the final architecture documentation.

8. Conclusion:

   This SLE helped me understand the architecture of the BFS and DFS search system using the C4 model. The Context level shows the overall system interaction, while the Container and Component levels show its internal structure. The Code level identifies the main functions used in the system. The C4 model makes the system easier to understand and explain.